

RurTech.ai — CDSCO Regulatory Workflow Automation

Official Hackathon Project Report

Team: RurTech.ai

Challenge: CDSCO Regulatory Workflow Automation Platform

I. Model Code and Documentation

1. Key Methodology and Model Development

Our platform is built on a decoupled, edge-optimized architecture leveraging **NVIDIA NIM Microservices** to ensure data sovereignty, extreme inference speed, and high accuracy.

- **Core AI Engine:** We utilize `meta/llama-3.1-70b-instruct` and `nvidia/llama-3.1-nemotron-70b-instruct` via the NVIDIA API for all reasoning tasks. These models were selected for their superior instruction-following capabilities, which are crucial for adhering to strict CDSCO regulatory formats.
- **Modular Pipeline:** The system is divided into specialized micro-agents: `anonymiser.py`, `summariser.py`, `assessor.py`, and `classifier.py`. Each agent operates with heavily engineered, zero-shot system prompts explicitly grounded in CDSCO nomenclature (e.g., NDCT Rules 2019, SUGAM portal standards).
- **Knowledge Retrieval (RAG):** We implemented a Retrieval-Augmented Generation (RAG) engine utilizing BM25 keyword matching over an extracted corpus of 11 official CDSCO PDF manuals, ensuring the AI responses are legally grounded rather than hallucinated.

2. GitHub Access Instructions

As requested by the IndiaAI Hackathon 2025 guidelines, access has been granted to the evaluation team.

To verify or grant access manually:

1. Navigate to the repository on GitHub.
2. Click the **Settings** tab.
3. In the left sidebar, click **Collaborators**.
4. Click **Add people** and search for `indiaaihackathon25`.
5. Add as a collaborator with Read/Write access.

(Note: The GitHub source code link has been copied to the official application form).

II. Project Report: Key Findings and Analysis

a. Detection Methodology

Our approach utilizes **Abstractive NLP and Structured Data Extraction**.

- **Completeness Verification:** We use a hardcoded, rule-based 26-field SUGAM mandatory checklist (defined in `config.py`). The LLM is instructed to extract all found entities into a JSON schema. The backend Python logic performs a strict set intersection between the extracted keys and the mandatory checklist. Anything missing is appended to a `deficiencies` array.

Version Comparison: We employ a semantic comparison engine. Instead of a simple `git diff` (which flags trivial formatting changes), the LLM analyzes `Version A` and `Version B` to detect substantive* medical or regulatory changes (e.g., altered dosage, changed active pharmaceutical ingredients), ignoring whitespace or synonym swaps.

b. Anonymisation Report

Compliance with the DPDP Act 2023 is achieved via a strict two-step process:

1. **De-identification & Pseudonymisation:** The LLM identifies specific PII/PHI categories (Names, Aadhaar, Phone, Medical Conditions) and replaces them with format-preserving tokens (e.g., `[PATIENT_NAME_1]`).
2. **Irreversible Generalisation:** For extreme privacy, specific dates (e.g., `14-Aug-2023`) are generalized to `[Q3-2023]`, and ages (42) are generalized to `[40-50]`.

Sample Output:

Raw:* "Patient John Doe, Aadhaar 1234-5678-9012, suffered cardiac arrest after taking Ergotamine."

Anonymised:* "Patient `[PATIENT_1]`, Aadhaar `[REDACTED_ID]`, suffered `[ADVERSE_EVENT_1]` after taking Ergotamine."

Backend Logic:* Generates a SHA-256 Blockchain Hash of the transaction to prove the token mapping hasn't been tampered with.

c. Flagging Mechanism & Duplicate Detection

- **Flagging:** Missing application fields automatically trigger a "Reviewer Action Required" flag in the backend, which is rendered as a high-visibility ■ RED box on the frontend UI.
- **Duplicate Detection:** When processing SAE (Serious Adverse Event) reports, the system extracts a unique signature: `Hash(Age + Gender + Suspect Drug + Event Date)`. If a new submission matches an existing signature in the database above an 85% confidence threshold, it is flagged as a duplicate.

d. Classification Criteria (Severity)

SAE classification is explicitly mapped to CDSCO guidelines. The AI scans the case narrative and enforces the following triage matrix:

- **Death / Life-Threatening:** Triage Priority CRITICAL
- **Permanent Disability / Congenital Anomaly:** Triage Priority HIGH
- **Prolonged Hospitalisation:** Triage Priority MODERATE
- **Other Medically Significant Events:** Triage Priority LOW

e. Specific Strategy for Distinct Source Materials

1. **Application Data (Checklists):** Treated as highly structured data. The strategy is pure extraction-to-JSON. No creative summarization is allowed.
2. **SAE Case Narration:** Treated as unstructured medical text. The strategy employs abstractive summarization to distill chaotic timelines into a strict `Onset -> Reaction -> Outcome` format.
3. **Meeting Transcripts / Audio:** The system utilizes Speech-to-Text (STT) followed by an NLP extraction strategy focused entirely on isolating "Action Items," "Decisions Made," and "Pending Approvals," stripping away conversational filler.

f. Standardised Format & Prioritisation

- **Output Format:** Every AI interaction returns a standardized JSON payload, coupled with an NLP-synthesized "Plain English" conclusion.

- **Prioritisation Logic:** The platform calculates a Triage Score. Applications with zero missing fields but a CRITICAL SAE severity are floated to the top of the reviewer's queue for immediate action. Applications with >50% missing fields are automatically queued for rejection without requiring human review time.